

Aoun Al-Darb Specialized Transport

What was built, what was actually measured against it, and what has not been measured yet — using numbers that came out of an execution rather than out of memory.

GO WITH ACTIONS

2,026 checks • 10 gates • 0 failures

The release gate (GATE 03) clears the package for upload, with three actions that are performed on the host and cannot be verified from the build environment. There is no open defect that is known and unfixed.

10

dashboard modules, every one reading and writing through the API

39

endpoints, each behind a declared permission

3

roles: Super Admin, Admin, Content Manager

0

runtime dependencies on the server — no Composer, no npm

What was measured

Ten gates, each run by one command and each returning a number. They are not reports written once — they are re-run before every release, and each was proven to fail on the real defect before it was trusted.

Output of php bin/release.php — 12 September 2026, 13:52 UTC

GATE	WHAT IT ASKS	RESULT	TIME
verify	Every endpoint through the real database	✓ 1001/1001	12s
authz	35 routes × 3 roles, against an oracle independent of the route table	✓ 105/105	1s
security	The questions an attacker asks, actually executed	✓ 42/42	4s
migrate	The schema on MySQL, and that re-migrating changes nothing	✓ 24/24	1s
browser	Every page in a real browser, at four widths	✓ 432/432	166s
controls	347 controls — no button bound to nothing	✓ 15/15	148s
public	The public site: head, form, 404 page	✓ 29/29	9s
ux	Failure, emptiness, dialogs, error messages	✓ 112/112	62s
ally	Contrast, focus, 320px, 200% zoom	✓ 181/181	294s
perf	Weight, paint, images, endpoint timings	✓ 85/85	47s

Why these numbers count for something: every gate was broken on purpose before it was trusted. A real SQL injection was introduced and the security gate caught it with the evidence a reader needs — ' OR '1'='1 returned 112 rows where a literal string carrying the same digits returned 8. HttpOnly was dropped from the session cookie and it caught that too. A gate that does not fail on the defect proves nothing about its absence.

Security — the mechanism, and what proves it

Naming the mechanism is not enough; this is what is actually executed to verify each one on every release.

Passwords	Argon2id via <code>password_hash</code> , with automatic re-hashing if the parameters change. No password is ever displayed or emailed.
Sessions	Rows in the database, not signed cookies. The primary key is the SHA-256 digest of the token — a copy of the database is not a set of usable sessions. The token rotates at sign-in (no session fixation), and signing out invalidates it on the server, not in the browser.
Session lifetime	120 minutes idle, and a 12-hour absolute ceiling regardless of activity. An aged token was actually refused during the check.
Session cookie	<code>HttpOnly</code> · <code>SameSite=Lax</code> · <code>Secure</code> · <code>Path=/</code> — all four read off the response, not off the source.
CSRF	Double-submit token compared with <code>hash_equals</code> . The gate sends a request with no token, with a fabricated token, <i>and with a valid token issued to a different session</i> — all three are refused with 419.
Authorization	Enforced in the dispatcher, never in the interface. A 35-route × 3-role matrix checked against an oracle independent of the route table — the first version read from the table it was testing, so 104 checks passed while a Content Manager was reading the reports.
SQL injection	Every query is parameterised. Seven payloads are measured against a literal string carrying the same digits — no difference in the result means they are data, not instructions.
Markup injection	The critical place is the publish pipeline, which writes HTML from operator input. An <code></code> tag in a service title does not reach <code>index.html</code> as executable markup.
Unexposed fields	<code>rejectUnknown</code> on every endpoint. Sending <code>status</code> or <code>id</code> with a public request rejects the whole submission (422), and a Content Manager cannot promote themselves.
Uploads	A PHP file named <code>.webp</code> is refused · an SVG carrying a script is refused · a valid image with code appended is refused · and <code>/img/</code> executes nothing.
Credential guessing	Account lock after 8 failures for 15 minutes, plus a limit of 10 attempts per address per 15 minutes. The refusal message is identical whether the account exists or not.
Public form	8 submissions per address per hour · a hidden honeypot field · a 10-minute duplicate window · any field outside the list rejected.
Secrets	110 servable files in the package were scanned for key material — none found. <code>.env</code> never ships, and <code>/app/</code> and <code>/bin/</code> answer 404.
Errors	No server path and no stack trace reaches a user. Even the browser's own <code>Failed to fetch</code> is translated into a sentence that says what to do.

Security headers that ship

Written into `.htaccess` inside the package: a Content-Security-Policy that names the inline scripts by hash rather than by `unsafe-inline`, plus `Strict-Transport-Security`, `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, and a `Permissions-Policy` that closes camera, microphone, geolocation and payment. Permanent redirects from `http` to `https` and from `www` to the bare domain. The whole dashboard is `noindex`.

Performance — measured on the built package

The public page is measured from `dist/` rather than from source, because the build rewrites it: 417 KB of source becomes 202 KB of package.

Uncompressed weight · LCP and CLS from the browser's own observers					
PAGE	REQUESTS	WEIGHT	DOCUMENT	LCP	CLS
index.html (public)	16	518 KB	202 KB	172 ms	0.001
Home	18	485 KB	89 KB	104 ms	0.086
Requests	17	539 KB	126 KB	72 ms	0.015
Customers	17	522 KB	107 KB	80 ms	0.022
Services	25	562 KB	123 KB	100 ms	0.009
Content	17	493 KB	117 KB	76 ms	0.022
Media	30	605 KB	135 KB	148 ms	0.022
Reports	18	506 KB	112 KB	68 ms	0.006
Users	16	831 KB	118 KB	72 ms	0.002
Settings	17	520 KB	159 KB	92 ms	0.092
Activity log	17	507 KB	110 KB	88 ms	0.015
Sign-in	9	301 KB	23 KB	52 ms	0

The "good" thresholds in Core Web Vitals are $LCP \leq 2,500$ ms and $CLS \leq 0.1$. Every page is under both. The slowest endpoint is the district coverage report at 112 ms; the rest are under 15 ms.

What the performance gate fixed

The dashboard had nothing small enough to draw with. The smallest rung of the public image ladder is 360 px, because that is the narrowest a service card is ever painted on the site — but the Media library draws 202×152 tiles and the Services list draws 84×60 rows, so both were sending the master file. A 5041×3577 logo was being downloaded in full to draw one tile.

- **Services** 1,602 KB → 562 KB
- **Media** 1,063 KB → 605 KB, and LCP 984 ms → 148 ms
- **Settings** CLS 0.149 → 0.092 — the page was painting in the fallback typeface and then reflowing every field 14 px upward when the real face arrived

Accessibility

181 measurements across twelve screens and the public site: the contrast ratio of every piece of text against the surface actually behind it, a complete keyboard walk driven by real Tab presses, heading and landmark structure, 320 px width, 200% zoom, and respect for reduced-motion preferences.

- Three real contrast violations, fixed using colours from the approved palette itself: navigation group labels 4.30:1 → 5.22:1, muted text on the public site 3.03:1 → 5.85:1, and the active navigation link 4.35:1 → 8.01:1.
- 35 error messages were drawn but never announced — tied to their fields with `aria-describedby` and `aria-invalid`.
- Three upload buttons the keyboard could not reach — they were a `<label>` over a file input hidden at one pixel.

- **Advisory, not a failure:** 40×40 and 36×36 controls at phone width. These clear the WCAG AA requirement (24 px) comfortably and fall short of the 44 px guideline. A deliberate desktop density.

What was not measured — and this is the boundary

Everything above ran on a local development server, over plain `http`, on SQLite. This build environment is walled off from the public internet, so nothing in this report is a measurement of `aunaldarb.com` itself.

The TLS layer

The certificate, HSTS as actually delivered, and the `http→https` redirect have never been exercised. The rules exist in `.htaccess`, but a rule existing is not a rule arriving.

LiteSpeed, not Apache

Hostinger does not run Apache. LiteSpeed reads `.htaccess` but not identically. The headers, compression and caching are **asserted from the file, not observed on the host**.

The real MySQL

24 schema checks ran against a local MariaDB — not against the Hostinger database, not with the real data, and not at its volume. The coverage report was timed on a small dataset.

Outside the scope of any gate

Hosting infrastructure, denial-of-service attacks, the security of the Hostinger control-panel account itself, and social engineering. None of these can be tested from inside the application.

Before upload — four actions

1

Delete
`install.php`
after
extracting

The package contains it and `recover.php`. Both answer 404 without a token in `.env` — but both reappear every time the package is extracted.

2

Confirm
`.env`
on the
host

`SESSION_COOKIE_SECURE=true · DB_DRIVER=mysql · APP_DEBUG=false` · and no `SETUP_TOKEN` or `RECOVERY_TOKEN`.
Better still, keep the file above `public_html` rather than inside it.

3

Take a
backup
before
uploading

From Settings → Security and Backup, or `php bin/backup.php`. Restoring is documented in the operator's manual.

It is what measures the four limits above on the host itself: the certificate, the headers as LiteSpeed actually sends them, MySQL, and the state of `.env`.

```
php bin/preflight.php --url=https://aunaldrb.com
```

Do not run `bin/release.php` against the live site. It invokes `verify.php`, which writes scratch data and publishes into `index.html`; it refuses to run in a production environment for exactly that reason. `preflight.php` is entirely read-only and is safe against the live site as often as you like.

Two open decisions — neither is a defect

- **Google Tag Manager** on the public page. The only external script in the entire project, and it sends visitor data to Google. The Content-Security-Policy permits it explicitly by name. Keeping or removing it is your decision, and removing it is one line.
- **The Ministry of Health licensing question** and the two written questions to the Transport General Authority remain open from the discovery phase. Neither has anything to do with the software.

Where the WEBSTART framework stands

Of twenty phases and three gates, all are complete except Phase 20, which is waiting on data rather than on code.

- **Phases 00–18 and all three gates** — complete. Their outputs are five delivered documents: the operator's manual, the commercial map, the system architecture, the master prompt, and the usability session observation sheet.
- **Phase 20 — post-launch optimisation** — waiting on two weeks of recorded WhatsApp requests, then the list of unmatched addresses to feed the district coverage report.

Branch `claude/aun-aldrb-brand-identity-7by0a9` Package 5.7 MB · 174 files Database 20 tables

Every figure on this page came out of an actual execution on 12 September 2026, not an estimate. What was not measured is named by name in its own section.